AIQA — Consolidated PRD + FRD + SRD

Version 6.0 — Hybrid Multi-Agent QA Architecture

Update: This version extends the existing v5.0 Mobile Testing baseline with the recommended Hybrid Multi-Agent + Central Orchestrator + Common Execution Engine architecture. Existing project-centric, real-time, evidence-driven and fault-tolerant requirements remain retained. ■filecite■turn4file2■

Primary quality focus: Edge Cases • RBAC • Security/Penetration • API Latency • UI/UX • Mobile. ■filecite■turn4file0■

PART A — PRODUCT REQUIREMENTS DOCUMENT (PRD)

1. Product Definition

AIQA is a project-centric AI quality engineering platform that ingests project context, builds a living quality model, generates and validates structured tests, routes work through a central AIQA Orchestrator to specialized testing capabilities, executes through a common execution platform, streams results in real time, investigates failures, preserves evidence and produces reports. The platform is not limited to test-case generation. ■filecite■turn4file0■

2. Product Modules

Module	Purpose
Dashboard	Project-wide health, execution/pass-fail/test/report cards, pie/bar analytics and project selector.
Test Case Generator	Ingest project evidence and generate categorized positive/negative tests in one unified view.
AIQA Orchestrator	Plan, route, prioritize and coordinate specialized test intelligence and execution.
Specialized Test Intelligence	Functional, Non-Functional, Edge, Scenario, Business Logic, UI/UX, Security, Penetration, RBAC, API, Performance, Databa
Test Case Validator	Traceability, quality threshold, duplicate, coverage and risk validation before execution.
Test Execution Hub	DAG scheduling, queues, workers, dependencies, locks and live execution.
RBAC / API / Mobile / Custom	Dedicated project-scoped testing experiences as defined in the existing baseline. ■filecite■turn4file2■
Reports / Settings / AI Model	Full or module reports, configuration, and local/API model abstraction. ■filecite■turn4file2■

3. Hybrid Multi-Agent Architecture

• One central AIQA Orchestrator rather than one independent model per testing type.

• Requirement Intelligence: understands FRD/SRD/PRD/Figma/API/DB/repository context and feeds the Quality Graph.

• Test Design Intelligence: Functional, Non-Functional, UI/UX, Edge, Scenario, Business Logic and Regression.

• Security Intelligence: Security, Penetration, RBAC, authentication and authorization.

• API/Performance Intelligence: API contracts, latency, performance and load-oriented test design.

• Mobile Intelligence: platform/device-aware test selection and planning.

• Test Case Validator: traceability, duplicate detection, coverage, risk and quality gates.

• Execution Orchestrator: DAG, queue, worker dispatch, resource/device locks and dependency handling.

• Evidence/Result Intelligence: evidence correlation, failure classification, RCA hypotheses, flakiness and reporting.

4. Agent ≠ Executor

Agents decide what/why/priority and validate. **Executors** perform actual browser/API/DB/security/mobile actions. This separation is the recommended architectural rule so model changes do not require executor rewrites.

5. Model Gateway

All agents shall use an AI Model Gateway rather than hard-coding a provider. The gateway supports local inference plus API fallback and runtime telemetry. Model choice can consider task type, context size, latency, availability and configured cost. The existing baseline already requires local/API abstraction and benchmark-based accuracy measurement. ■filecite■turn4file4■turn4file3■

Important: 90%+ useful test-generation accuracy is a measured target, not a guarantee. Generated tests must pass quality gates before automatic execution. ■filecite■turn4file1■

PART B — FUNCTIONAL REQUIREMENTS DOCUMENT (FRD)

6. Orchestrator & Agent Requirements

• FRD-AGT-01: Create a project-scoped test/analysis plan.

• FRD-AGT-02: Route tasks to the appropriate specialized capability.

• FRD-AGT-03: Share project_id and approved application/quality context across capabilities.

• FRD-AGT-04: Validate structured agent output before automatic execution.

• FRD-AGT-05: Prioritize using risk, change impact, test value, dependencies and execution cost.

• FRD-AGT-06: Isolate an agent failure from unrelated work where possible.

• FRD-AGT-07: Record routing/decision metadata sufficient for audit.

• FRD-AGT-08: Never treat unsupported AI conclusions as confirmed defects.

7. Test Case Generation & Validation

The existing unified generator supports Functional, Non-Functional, UI/UX, Edge Case, Scenario-Based, Business Logic, Security, Penetration, RBAC, Regression, Database and API testing, including positive and negative cases. ■filecite■turn4file0■

• FRD-VAL-01: Critical requirements map to at least one test.

• FRD-VAL-02: Generated cases meet configured quality/benchmark rules.

• FRD-VAL-03: Duplicate or overlapping cases are detected.

• FRD-VAL-04: Edge, business, RBAC and security coverage is checked against risk.

• FRD-VAL-05: Each case retains category, priority, preconditions, steps, expected result and traceability.

8. Execution Orchestrator & Workers

• FRD-EXE-11: Convert approved test cases into schedulable tasks.

• FRD-EXE-12: Use dependency-aware DAG scheduling.

• FRD-EXE-13: Independent tests continue after unrelated failures.

• FRD-EXE-14: Dependent tests become BLOCKED when prerequisites fail.

• FRD-EXE-15: Distinguish product failures from environment/infrastructure failures.

• FRD-EXE-16: Support Playwright + Chromium visible execution.

• FRD-EXE-17: Support API, DB, Security and Mobile specialized workers.

• FRD-EXE-18: Support controlled parallelism for large suites, including the discussed ~1,000-test target. ■filecite■turn4file6■

• FRD-EXE-19: Use resource/device locks for unsafe concurrent mutations.

9. Real-Time, Evidence & Failure Intelligence

• FRD-RT-01: Normalize worker events.

• FRD-RT-02: Update dashboards incrementally.

• FRD-RT-03: Reconcile authoritative state after reconnect.

• FRD-RT-04: Link evidence to run/task/test/finding/report.

• FRD-FI-01: Correlate execution evidence, environment data, historical failures and logs for RCA.

• FRD-FI-02: Uncertain AI findings remain anomaly/needs-review, not confirmed defects. ■filecite■turn4file7■

10. Existing Mobile Requirements Retained

The v5.0 Mobile Testing module remains: dedicated dashboard, Samsung/Android and Apple/iOS selection, device inventory/capabilities, Test Case Generator integration, UI/UX exclusion, device locks, real-time events and evidence. ■filecite■turn4file8■turn4file9■

PART C — SOFTWARE / SYSTEM REQUIREMENTS DOCUMENT (SRD)

11. Logical Architecture

Layer	Components
Frontend	Dashboard, Generator, Execution, RBAC, API, Mobile, Custom, Feature/Flow, Reports, Settings
Control Plane	Project context, auth/RBAC, policy, run/task APIs, authoritative state
AI Layer	Model Gateway, Orchestrator, Requirement Intelligence, Test Design, Security, API/Performance, Mobile
Validation	Traceability, quality gates, duplicate detection, risk/change/test-value checks
Execution	DAG Scheduler, Queue, Worker Pool, resource/device locks
Workers	Playwright/Chromium, API/HTTP, DB, Security/Penetration, Maestro/Appium
Data	PostgreSQL, semantic/vector retrieval, evidence storage, Redis/event transport
Observability	OpenTelemetry, logs, metrics, traces, WebSocket, audit

12. Agent Contract

Input: project_id, approved context references, task type, target scope, constraints and policy/capability context. **Output:** structured test/analysis/plan with source references, validation status and evidence/confidence metadata. Large raw artifacts shall not be sent wholesale through the LLM; targeted context shall be used. ■filecite■turn4file5■

13. Common Execution Contract

All executors shall normalize outcomes as QUEUED, RUNNING, PASSED, FAILED, BLOCKED, SKIPPED, CANCELLED, NEEDS_REVIEW and infrastructure/environment error, with screenshots, traces, logs, request/response and timing evidence where applicable. ■filecite■turn4file6■

14. Model Routing Requirements

• SRD-MDL-01: Agents call the Model Gateway, not providers directly.

• SRD-MDL-02: Gateway supports local model runtime and API fallback.

• SRD-MDL-03: Routing considers task type, context, latency, availability and cost policy.

• SRD-MDL-04: Gateway exposes model/runtime/latency/usage telemetry where available.

• SRD-MDL-05: Model/provider failure results in controlled fallback or review state; never fabricated output.

15. Performance & Real-Time

• Long-running tests never block API request threads.

• Dashboard updates incrementally.

• Concurrency is configurable per executor/environment.

• API latency supports p95/p99 where appropriate.

• Queue depth, task latency, event lag, worker failures and dashboard delivery lag are observable. ■filecite■turn4file5■

16. Security & Governance

• Server-side RBAC remains mandatory.

• Security, destructive DB, production load and external side-effect actions require explicit policy/authorization.

• Secrets are runtime references, not embedded in tests/logs.

• Security executors are isolated.

• Approvals, policy decisions, agent actions and side effects are auditable. ■filecite■turn4file5■

17. Persistence & Recovery

• PostgreSQL remains the durable system of record.

• Run/task/event state persists for recovery and audit.

• Evidence metadata is queryable without loading large binaries.

• Event processing is idempotent.

• Worker/API restart restores durable task/run state.

• Worker crash does not terminate unrelated tasks.

18. Acceptance Scenarios — v6.0

• Orchestrator routes Edge, RBAC, Security, API and Mobile work to specialized capabilities.

• Model gateway failure triggers controlled fallback or NEEDS_REVIEW.

• Generated tests pass quality/traceability gates before execution.

• 1000 independent tests execute through controlled workers while DAG dependencies are honored.

• RBAC live view shows the currently active role/access path.

• API view shows request/response and latency in real time.

• Browser execution is observable in Chromium.

• Mobile selects a compatible device, locks it, excludes UI/UX cases and records evidence. ■filecite■turn4file9■

• Full and module-specific reports reconcile with durable execution state.

19. Version Change Log

Version	Update
v4.0	Consolidated quality-intelligence and execution baseline. ■filecite■turn4file0■
v5.0	Added Mobile Testing Dashboard and mobile execution requirements. ■filecite■turn4file2■
v6.0	Added Hybrid Multi-Agent architecture, central Orchestrator, specialized testing intelligence, Test Case Validator, common Execution Orchestrato